



2025

MLOps ↗ Report

Milestone 4: MLOps,
Deployment, and
Monitoring

Table of Contents

01 Introduction

02 System Architecture Overview

03 Component Analysis

04 Model Training Pipeline

05 Summary of the MLOps Workflow

06 Conclusion

Introduction

This report explains the MLOps workflow implemented for the Sales Forecasting and Demand Prediction project.

The goal of the system is to predict future sales and future demand (quantity) using machine-learning models trained on historical transaction data.

The MLOps pipeline ensures:

- Reliability and traceability
- Automated experiment tracking
- Reproducible training
- Consistent deployment
- Scalable prediction API
- Monitoring and future retraining

“

System Architecture Overview

2025

The MLOps pipeline consists of **three main components**:

1. MLOps Experiment Tracking (MLops.py)
2. API Service Layer (api_service.py)
3. Web Dashboard Interface (Streamlit.py)

Component Analysis

1. MLOps Experiment Tracking (MLops.py)

- **Purpose:** Model management, experiment tracking, and performance monitoring using MLflow.
-

Key Functionalities:

- **Model Registry:** Manages 12 different machine learning models for sales and quantity prediction
- **Experiment Tracking:** Separates experiments into "Sales-Models" and "Quantity-Models"
- **Performance Metrics:** Tracks RMSE, MAE, R^2 , and MSE for model evaluation
- **Artifact Management:** Stores model visualizations and performance plots
- **Model Signatures:** Automatically infers input/output schemas for deployed models

DEBI

Supported Models:

- XGBoost (primary production models)
- Random Forest
- Decision Trees
- K-Nearest Neighbors
- AdaBoost

MLflow Integration:

- Sets tracking URI to local mlruns directory
 - Automatic model registration with versioning
 - Parameter logging for model reproducibility
 - Visual artifact storage for model diagnostics
-

2. API Service Layer (api_service.py)

- **Purpose:** RESTful API for model serving with feature engineering and prediction capabilities.

FastAPI Framework:

- **Automatic OpenAPI documentation**
- **Pydantic data validation**
- **Async capability support**
- **Comprehensive error handling**

Feature Engineering:

- Temporal feature calculation (lags, rolling statistics)
- Date-based feature generation (weekend, month boundaries)
- Dynamic profit calculation using historical margins
- Intelligent caching for performance optimization

Model Serving:

- Loads production XGBoost models for sales and quantity prediction
- Ensures non-negative predictions
- Comprehensive input validation
- Health check endpoints for monitoring

Data Flow::

- Receives prediction request with business parameters
 - Calculates temporal features from historical data
 - Constructs feature vector matching training schema
 - Returns predictions with feature metadata
-

3. Web Dashboard Interface (Streamlit.py)

- **Purpose:** User-friendly interface for business users to interact with the prediction system.

Streamlit Framework:

- Responsive web application
- Real-time interaction
- Custom CSS styling for professional appearance

Input Categories:

- General Inputs: Date, discount rates, shipping costs, seasonal factors
- Product Details: Category, sub-category, shipping methods
- Customer & Location: Market regions, geographic segments, customer types

Model Training Pipeline

2025

The training pipeline is built using:

- **ColumnTransformer** : Handles numeric + categorical features automatically.
- **XGBoost Regressor**: Chosen as best-performing model for both sales & demand.

2. API Service Layer (api_service.py)

- **Purpose**: RESTful API for model serving with feature engineering and prediction capabilities.

FastAPI Framework:

- **Automatic OpenAPI documentation**
- **Pydantic data validation**
- **Async capability support**
- **Comprehensive error handling**

Feature Engineering:

- **Temporal feature calculation** (lags, rolling statistics)
- **Date-based feature generation** (weekend, month boundaries)
- **Dynamic profit calculation** using historical margins
- **Intelligent caching** for performance optimization

Summary of the MLOps Workflow

Stage	Tools Used	Purpose
Data Processing	pandas, sklearn	Clean & prepare dataset
Experiment Tracking	MLflow	Track metrics and parameters
Model Training	XGBoost + Pipeline	Train reproducible models
Feature Engineering	Python auto-features	Lags & date features
Deployment	FastAPI	Real-time predictions
User Interface	Streamlit	Dashboard for business users
Caching	Python dict	Faster inference
Model Registry	MLflow	Version control
Monitoring	Logging	Drift detection & performance

Conclusion

This MLOps implementation provides a robust, scalable, and maintainable pipeline for sales forecasting and demand prediction. The separation of experiment tracking, model serving, and user interface allows for independent development and deployment while maintaining system integrity. The use of industry-standard tools (MLflow, FastAPI, Streamlit) ensures compatibility and extensibility for future requirements.
